SWEAT

Spatial Water stress Evapotranspiration Assessment for TRISHNA

Emmanuelle Sarrazin

2026-10-07

Overview

SWEAT contains L2B/L3 algorithms used in TRISHNA mission to compute evapotranspiration:

- **EVASPA** EVApotranspiration from SPAce
- **STIC** Surface Temperature Initiated Closure
- **ET time series**

Pre-requisites

- git
- pixi, see instruction for installation here.

Installation

Clone the repository

```
git clone https://github.com/CNES/sweat.git
```

Install

```
cd sweat/
pixi install
```

Install notebooks dependencies

```
pixi shell
pip install .[notebook]
```

Update

```
cd et-dataset
git pull
pixi shell
pip install .[notebook]
```

Developer installation

Install in a developer mode (dev tools, notebooks, docs)

```
pixi install -e dev
```

Update in developer mode

```
cd et-dataset
git pull
pixi shell -e dev
```

Usage: Command line

Activate the environment

```
pixi shell [-e dev]
```

Command line

4 main commands available

- evaspa
- stic
- timeseries
- window_timeseries (timeseries over a period)

Usage: High-level API

Provide by module `sweat.api`

- run_evaspa
- run_stic
- run_timeseries
- run_window_timeseries

Usage: Notebooks

Provide examples to use API

Activate the environment

```
pixi shell
jupyter-lab
```

Notebooks

- Main notebooks located in the root of notebooks directory
- More detailed notebooks in subdirectories of notebooks directory

Documentation generation

Installation

```
pixi shell
pip install -e .[docs]
```

Build the documentation

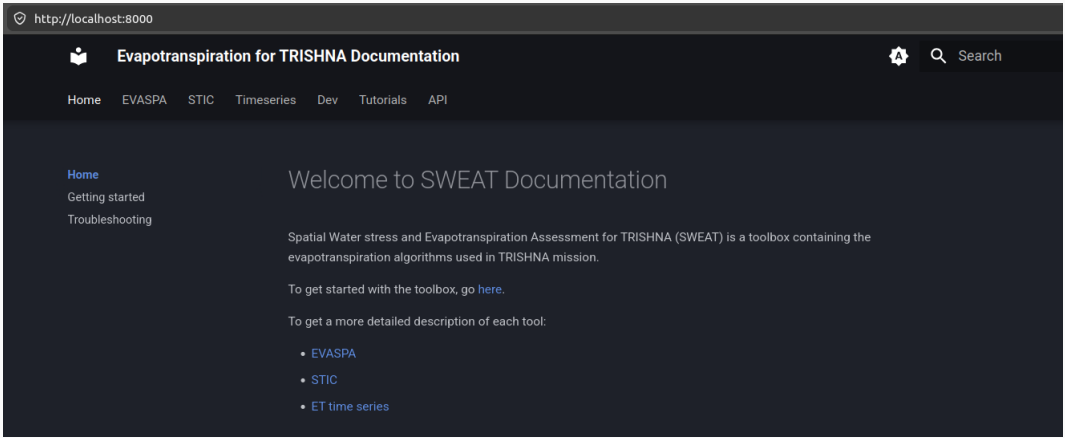
```
mkdocs build
```

Start the live-reloading docs server

```
mkdocs serve
```

Documentation visualization

Open http://localhost:8000/



File tree structure

```
|-- AUTHORS.md
|-- CONTRIBUTING.md
|-- docs
|-- hooks
|-- LICENSE
|-- Makefile
|-- mkdocs.yml
|-- notebooks
|-- pyproject.toml
|-- README.md
|-- ruff.toml
|-- src
|-- tests
`-- troubleshooting.md
```

Module list

Module	Description
`api.py`	High-level API
`cli`	Module for CLI management
`common`	Module containing common submodules
`debugging.py`	Module for debugging management
`evaspa`	Module containing EVASPA submodules
`logging.py`	Module for logging
`misc`	Module containing auxiliary data submodules
`stic`	Module containing STIC submodules
`timeseries`	Module containing timeseries submodules

Configuration Management

- Stored in *JSON* format (*YAML* format also possible)
- Managed via `Pydantic`
  - validate the configuration file and configuration dictionary:
    - Verify that the required parameters are present.
    - Verify that the parameters conform to the expected format.
    - Verify that the parameters conform to the expected values.
  - Handle default parameter values, so the code can be run with very few parameters specified

$\Rightarrow$ **Objective** To perform an early configuration check to avoid a later error during execution.

Configuration Management

- The complete configuration used for each SWEAT run is written to the *config.json* file located in the output directory
- This file can then be used for a new execution of the code.

High-level API

- Use `xarray.Dataset` for data
- Use Python dictionaries for configuration

Data

Remark on inputs

- SWEAT uses the band descriptions during input reading to determine which variable each band corresponds to.
- It is essential to correctly write the band name in the description field and to ensure that the band names match the expected format.
- SWEAT is compatible with et-dataset.

STIC

STIC Algorithm steps



Usage: Command line

```
Usage: stic [OPTIONS] INPUT_FILE

Options:
  --verbose / --no-verbose  Debug logging mode
  --version                 Show the version and exit.
  -h, --help                Show this message and exit.
```

Input file description

- **input**: provide information for input data
- **output**: provide information to write results
- **params**: provide parameters to processing steps
- **debug** (optional): provide information for debug mode

Usage: Command line

An example of input file:

```
{
    "input":{
        "path": "tests/data/data_test_20230303T112730.tif",
        "date": "2023-03-03T11:27:30-00:00"
    },
    "output":{
        "path":"out"
    }
}
```

Output description

- a directory **stic_daily** which contains daily products: LE, ET and flags
- a directory **stic_inst** which contains instantaneous products: EF, LE, ET and flags
- a directory **stic_waterstress** which contains water stress products: water stress and flags
- a file **config.json** which contains the detailed configuration used
- a directory **debug** if debug mode is activated

Usage: High-level API

```
def run_stic(
    data: xr.Dataset, params: dict, debug: dict | None = None
) -> tuple[xr.Dataset, xr.Dataset, xr.Dataset] | None:
```

Inputs

- **data** input data
- **params** parameters configuration
- **debug** debug configuration

Outputs

- Instant dataset: EF, LE, ET and flags
- Daily dataset: LE, ET and flags
- Water stress dataset: water stress and flags

Usage: Notebook

- Main notebook **run_stic.ipynb**

Input configuration

- Detailed in usage page and configuration page

Module list

Module	Description
`config.py`	Module for STIC configuration management
`constant.py`	Module for STIC constants
`convert.py`	Module containing functions to convert variables used to prepare data for STIC model
`main.py`	Module containing the API for STIC
`models`	Module containing STIC models
`registry.py`	Module containing register for STIC models
`runner.py`	Module containing methods for running STIC models

Software design (new version)

- Change in the structure of the **models** module

```
src/sweat/stic/models/
|-- flux.py
|-- functions.py
|-- __init__.py
|-- v1_3
|   |-- __init__.py
|   |-- model.py
|   `-- smwetness.py
`-- v1_4
    |-- __init__.py
    |-- model.py
    `-- smwetness.py
```

How to add a new model

- Create a new subdirectory in *models/*
- This directory must contain at least the **model.py** file
- The **model.py** file must contain the following elements

```
VERSION: str  # Version name
IS_DEFAULT: bool
MAPPING_VARIABLES: dict[str,str] # Mapping of model required variables
REQUIRED_INPUTS:list[str] # List of required input data
def run_stic_model(...) # run STIC for a raster
def run_batch_stic_model(...) # run SITC on a dataframe
def run_batch_init_model(...) # run STIC init step on a dataframe
```

How to run a model

- Automatic registration with **registry.py**
- Run with **runner.py**
  - `run_model`
  - `run_batch_model`
  - `run_batch_int_model`

How to run a model

run_model

```
def run_model(
  data: xr.Dataset,
  valid: npt.ArrayLike | None = None,
  threshold: float = 0.01,
  nb_steps: int = 15,
  version: str | None = None,
  precision: TYPES = "float32"
) -> tuple[npt.NDArray, npt.NDArray, npt.NDArray]:
```

How to run a model

run_batch_model

```
def run_batch_model(
  data: pd.DataFrame,
  threshold: float = 0.01,
  nb_steps: int = 15,
  version: str | None = None,
  debug: bool = False,
  mapping: bool = True,
  precision: TYPES = "float32"
) -> npt.NDArray:
```

How to run a model

run_batch_init_model

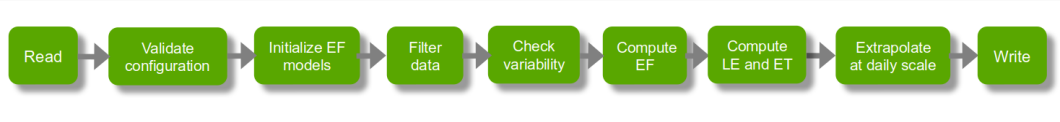
```
def run_batch_init_model(
  data: pd.DataFrame,
  version: str | None = None,
  debug: bool = False,
  mapping: bool = True,
  precision: TYPES = "float32"
) -> npt.NDArray:
```

Exercises

- Test **tutorial_stic.ipynb**
- Test **tutorial_stic_on_sites.ipynb**

EVASPA

Algorithm Steps



Usage: Command line

```
Usage: evaspa [OPTIONS] INPUT_FILE

Options:
  --verbose / --no-verbose  Debug logging mode
  --version                 Show the version and exit.
  -h, --help                Show this message and exit.
```

Input file description

- **input**: provide information for input data
- **output**: provide information to write results
- **params**: provide parameters to processing steps
- **debug** (optional): provide information for debug mode

Usage: Command line

An example of input file:

```
{
    "input":{
        "path": "tests/data/data_test_20230303T112730.tif",
        "date": "2023-03-03T11:27:30-00:00"
    },
    "output":{
        "path":"out"
    },
    "params":{
        "ef":{
            "models": "default_evaspa"
        }
    }
}
```

Output description

- a directory **evaspa_daily** which contains daily products: LE, ET and flags
- a directory **evaspa_inst** which contains instantaneous products: EF, LE, ET and flags
- a directory **evaspa_waterstress** which contains water stress products: water stress and flags
- a file **config.json** which contains the detailed configuration used
- a directory **debug** if debug mode is activated

Usage: High-level API

```
def run_evaspa(
    data: xr.Dataset, params: dict, debug: dict | None = None
) -> tuple[xr.Dataset, xr.Dataset, xr.Dataset] | None:
```

Inputs

- **data** input data
- **params** parameters configuration
- **debug** debug configuration

Outputs

- Instant dataset: EF, LE, ET and flags
- Daily dataset: LE, ET and flags
- Water stress dataset: water stress and flags

Usage: Notebook

- Main notebook **run_evaspa.ipynb**

Other notebooks ../notebooks/evaspa

- **edge_benchmark.ipynb**: Notebook to test edge configuration
- **ef_generic_model.ipynb**: Notebook to describe how edge model works
- **ef_models.ipynb**: Notebook to describe how EF model works
- **ef_processing.ipynb**: Notebook to describe EF processing
- **le_processing.ipynb**: Notebook to describe LE processing
- **evaspa_zones.ipynb**: Notebook to describe how to define EVASPA valid zones
- **evaspa_tiling.ipynb**: Notebook to describe how to merge EVASPA tiles

Configuration

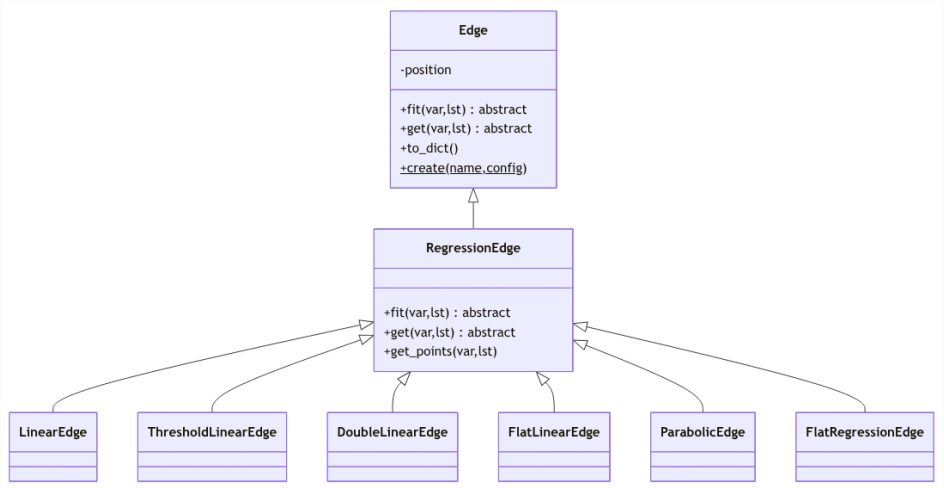
Input configuration

- Detailed in usage page and configuration page

EF models configuration

- Detailed in EF model page

Edge class


Edge
-position
+fit(var,lst) : abstract
+get(var,lst) : abstract
+to_dict()
+create(name,config)
RegressionEdge
+fit(var,lst) : abstract
+get(var,lst) : abstract
+get_points(var,lst)
LinearEdge
ThresholdLinearEdge
DoubleLinearEdge
FlatLinearEdge
ParabolicEdge
FlatRegressionEdge

Model class


EFModel
-name
-wet_edge
-dry_edge
-var
+fit(data,mask)
+tdry(var)
+twet(var)
+fit(data,mask)
+compute(data,mask)
+to_dict()
+check(config)
+create(config)

Module list

Module	Description
`config.py`	Module for EVASPA configuration management
`edge.py`	Module containing edges classes
`ef.py`	Module for Evaporative Fraction model management
`merging.py`	Module containing functions for merging datasets
`seb.py`	Module containing functions for Surface Energy Balance
`tiling.py`.py	Module for tiling management
`zones.py`.py	Module to manage valid zones for EVASPA ## Software design (new version)

Exercises

- Test **tutorial_evaspa_ef_model.ipynb**
- Test **tutorial_evaspa.ipynb**